INFORME DE SEGURIDAD

Informe de análisis estático de seguridad

Portal Vial y Educación Vial IMSJ

Fecha	20 de octubre de 2026
Herramienta	Semgrep OSS 1.180.0 con configuración automática
Alcance	144 archivos seguidos por Git en backend, frontends y documentación
Resultado bruto	11 coincidencias: 1 ERROR y 10 WARNING
Triage	1 debilidad confirmada repetida en 8 archivos y 3 alertas de backend mitigadas

El escaneo no confirmó una inyección ni un acceso arbitrario a archivos en el backend. La acción prioritaria es eliminar la dependencia remota sin verificación de integridad usada por ocho páginas públicas. Las alertas del backend deben conservarse como evidencia de revisión y pueden endurecerse para reducir ambigüedad en futuros análisis.

11	546	144	99,9 %
coincidencias	reglas ejecutadas	archivos analizados	líneas interpretadas

Equipo RC5

Clasificación interna | Entregable técnico

Resumen ejecutivo

Semgrep informó 11 coincidencias y las marcó como bloqueantes para su política de salida. La cifra no equivale a 11 vulnerabilidades explotables. Ocho coincidencias corresponden a una misma debilidad de suministro en páginas HTML y tres requieren interpretar controles del backend.

Resultado del triage	Coincidencias	Lectura
Debilidad confirmada	8	Script de unpkg sin atributo integrity en ocho páginas públicas
Mitigada por el código	2	Ruta de caché basada en hash y borrado restringido a UPLOAD_ROOT
No explotable en el flujo revisado	1	Comando de verificación CLI con argumentos escapados y archivos de un directorio fijo

Decisión recomendada

Antes de publicar el portal, hospedar localmente Lucide o fijar la versión con Subresource Integrity y crossorigin. Después, repetir Semgrep y conservar el JSON de revalidación. Las tres alertas del backend no justifican por sí solas bloquear la entrega, pero conviene refactorizar el verificador de sintaxis para evitar la invocación mediante shell.

Prioridad de trabajo

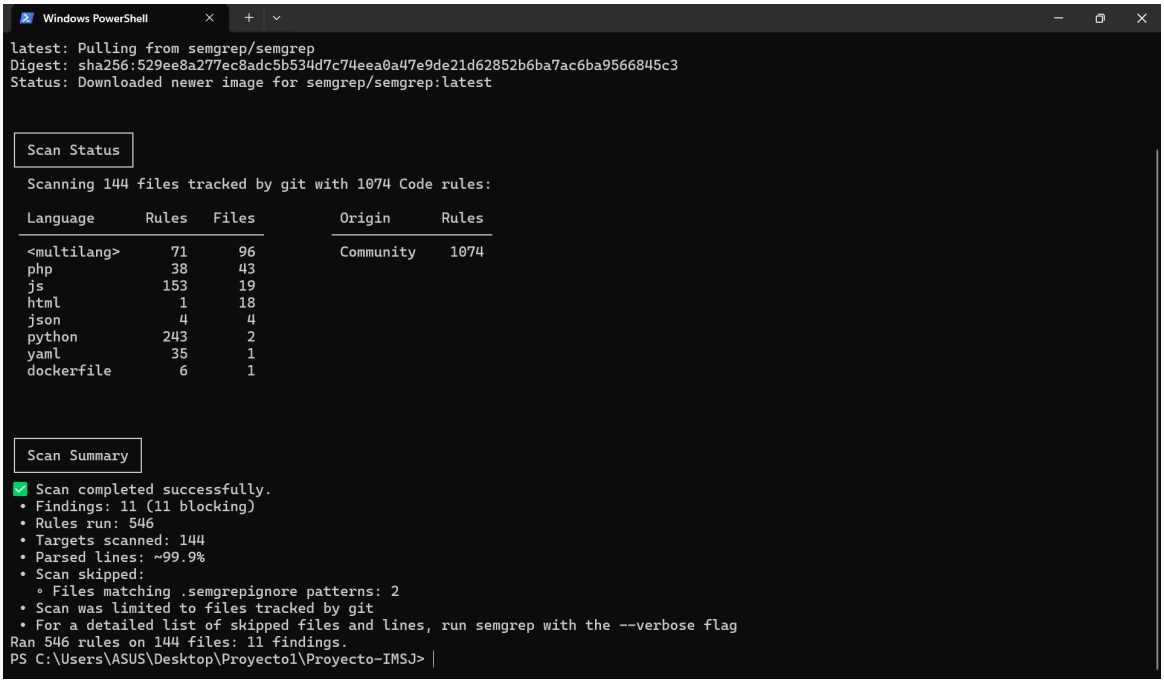
Orden	Acción	Criterio de cierre
1	Eliminar o verificar el script remoto en ocho HTML	Sin alertas missing-integrity y sin pérdida de iconos
2	Reemplazar passthru por una ejecución sin shell	El script verifica todos los PHP sin alerta exec-use
3	Documentar supresiones justificadas en Auth y Archivo	Excepción breve, localizada y asociada al control existente
4	Repetir el escaneo con salida fuera del repositorio	JSON sin error por autolectura y cobertura revisada

Alcance y método

El análisis se ejecutó con la imagen semgrep/semgrep y la configuración automática. La consola cargó 1.074 reglas comunitarias y aplicó 546 reglas compatibles a 144 archivos seguidos por Git. El reporte JSON identifica Semgrep OSS 1.180.0 y 11 resultados.

Lenguaje	Reglas mostradas	Archivos
Multilenguaje	71	96
PHP	38	43
JavaScript	153	19
HTML	1	18
JSON	4	4
Python	243	2
YAML	35	1
Dockerfile	6	1

Evidencia de ejecución



Captura 1. Resumen de la ejecución de Semgrep proporcionado con el análisis.

Cobertura y limitaciones

El resultado cubre el código que Semgrep alcanzó a analizar con las reglas aplicables. No sustituye pruebas dinámicas, revisión de configuración en ejecución, pruebas autenticadas ni análisis de dependencias. Un resultado sin alertas tampoco demostraría ausencia de vulnerabilidades.

Incidencia	Efecto	Tratamiento
sast-report.json produjo un error de sintaxis durante el escaneo	La redirección creó el archivo de salida dentro del volumen antes de que Semgrep terminara	Generar la salida fuera del repositorio o excluir ese nombre
Dockerfile tuvo análisis parcial entre las líneas 5 y 18	Parte de ese archivo pudo no recibir todas las reglas previstas	Revisar sintaxis y reanalizar el Dockerfile
Timeout de punto fijo en backend/public/index.php	Una regla de análisis de flujo no completó su evaluación	Repetir con mayor capacidad o una regla dirigida
Dos archivos ignorados y alcance limitado a Git	Archivos ignorados o no seguidos quedaron fuera	Revisar .semgrepignore y confirmar que el alcance sea intencional

Conclusión de cobertura. Los 11 resultados pueden analizarse, pero el informe no debe presentarse como una certificación integral del proyecto. La revalidación debe cerrar las incidencias de cobertura y demostrar la corrección del hallazgo confirmado.

Triage de hallazgos

ID	Regla y ubicación	Semgrep	Triage	Prioridad
SAST 01	missing-integrity en 8 páginas públicas	WARNING	Confirmado	Media
SAST 02	exec-use en backend/scripts/verificar.php:22	ERROR	No explotable en el flujo	Baja
SAST 03	unlink-use en backend/models/Archivo.php:80	WARNING	Mitigado	Baja
SAST 04	tainted-filename en backend/core/Auth.php:62	WARNING	Falso positivo	Informativa

Criterio aplicado

La severidad de Semgrep se conserva como dato de la herramienta. La prioridad del informe considera si la entrada es controlable por un atacante, qué validaciones se ejecutan antes del punto sensible y si el flujo está expuesto por HTTP o limitado a una utilidad local.

Distribución del resultado bruto

Severidad	Cantidad	Reglas
ERROR	1	php.lang.security.exec-use.exec-use
WARNING	10	tainted-filename, unlink-use y missing-integrity

SAST 01
Dependencia externa sin verificación de integridad

CONFIRMADO

Severidad de Semgrep: WARNING

Ocho páginas cargan la versión 0.468.0 de Lucide desde unpkg. La versión está fijada, pero las etiquetas script no incluyen el atributo integrity. Si el contenido remoto o su cadena de distribución fuese alterado, el navegador ejecutaría el JavaScript modificado con los permisos de la página.

Evidencia

Archivo	Línea	Recurso
frontend-publico/agenda.html	21	unpkg.com/lucide@0.468.0/dist/umd/lucide.min.js
frontend-publico/consulta.html	21	unpkg.com/lucide@0.468.0/dist/umd/lucide.min.js
frontend-publico/estudio.html	21	unpkg.com/lucide@0.468.0/dist/umd/lucide.min.js
frontend-publico/index.html	151	unpkg.com/lucide@0.468.0/dist/umd/lucide.min.js
frontend-publico/noticias.html	86	unpkg.com/lucide@0.468.0/dist/umd/lucide.min.js
frontend-publico/preguntas.html	21	unpkg.com/lucide@0.468.0/dist/umd/lucide.min.js
frontend-publico/prueba.html	32	unpkg.com/lucide@0.468.0/dist/umd/lucide.min.js
frontend-publico/renovagenda.html	25	unpkg.com/lucide@0.468.0/dist/umd/lucide.min.js

Clasificación

CWE-353 Missing Support for Integrity Check. OWASP A08 Software and Data Integrity Failures.

Corrección recomendada

Opción	Implementación	Observación
Preferida	Hospedar lucide.min.js dentro del proyecto y servirlo desde el mismo origen	Reduce la dependencia operativa del CDN
Alternativa	Agregar integrity con el hash del archivo exacto y crossorigin=anonymous	Actualizar el hash cuando cambie la versión

Validación

Abrir las ocho páginas, verificar que los iconos se rendericen y repetir Semgrep. El cierre exige cero coincidencias de missing-integrity o una excepción documentada si se adopta otra arquitectura.

Hallazgos del backend

SAST 02
Ejecución de un comando construido dinámicamente

NO EXPLOTABLE

Severidad de Semgrep: ERROR

La línea 22 usa passthru para ejecutar el verificador de sintaxis de PHP. Semgrep trata cualquier comando no constante como posible inyección. En este flujo, PHP_BINARY es una constante del intérprete, los archivos se enumeran bajo backend y tanto el binario como la ruta pasan por escapeshellarg. Además, el script termina si no se ejecuta por CLI. No se identificó una entrada HTTP o un argumento del usuario que alcance el comando.

```
passthru(escapeshellarg(PHP_BINARY) . ' -l ' . escapeshellarg($file->getPathname()), $status);
```

Riesgo residual y mejora

El uso de una cadena de shell mantiene una superficie innecesaria y dificulta el análisis automático. Reemplazarlo por proc_open con una lista de argumentos, o por una API de procesos que no invoque shell, conserva la función y elimina el patrón reportado.

SAST 03
Eliminación de archivo a partir de una ruta variable

MITIGADO

Severidad de Semgrep: WARNING

Archivo::delete recibe una ruta variable, pero rechaza URLs, resuelve la ruta real desde UPLOAD_ROOT, exige que permanezca dentro de ese directorio y confirma que el destino sea un archivo antes de llamar a unlink. Estos controles bloquean el recorrido de directorios que describe CWE-22 en el flujo revisado.

```
!unlink($resolved)
```

Riesgo residual y mejora

Conservar las comprobaciones juntas y agregar pruebas automatizadas con rutas ../, rutas absolutas, URLs y enlaces simbólicos. Si la alerta se suprime, hacerlo en la línea exacta e indicar que realpath y la comparación con UPLOAD_ROOT son el control compensatorio.

SAST 04
Nombre de archivo derivado de datos de la solicitud

FALSO POSITIVO

Severidad de Semgrep: WARNING

El nombre de la caché combina la operación y la IP, pero antes de crear el archivo aplica SHA-256. El resultado es una cadena hexadecimal de longitud fija. CACHE_ROOT proviene de la configuración del servidor y el código no realiza una solicitud de red, por lo que la clasificación SSRF de la regla no coincide con el comportamiento observado.

```
$file = fopen(CACHE_ROOT . '/' . $key . '.json', 'c+');
```

Tratamiento

Mantener el hash y la raíz de caché fija. Documentar una supresión localizada con la justificación anterior y una prueba que confirme que el nombre final solo contiene 64 caracteres hexadecimales seguidos de .json.

Plan de corrección y revalidación

Paso	Responsable sugerido	Trabajo	Evidencia
1	Frontend	Autohospedar Lucide o configurar SRI en ocho HTML	Diff y prueba visual
2	Backend	Eliminar la cadena de shell en verificar.php	Salida del verificador
3	Backend y QA	Agregar pruebas de rutas y nombre de caché	Pruebas automatizadas
4	QA	Repetir Semgrep y revisar errores de cobertura	Nuevo JSON y captura
5	Revisión interna	Comparar resultados y aprobar excepciones	Acta o registro de cierre

Comando de revalidación

```
$tempReport = Join-Path $env:TEMP 'imsj-sast-report.json'
docker run --rm -v "${PWD}:/src" semgrep/semgrep semgrep scan --config auto --exclude sast-report.json --json >
$tempReport
Copy-Item -LiteralPath $tempReport -Destination .\sast-report.json -Force
```

El archivo temporal evita que Semgrep intente analizar su propia salida mientras se está escribiendo. La exclusión mantiene el comportamiento correcto si ya existe una copia del JSON en el repositorio.

Criterios de cierre

Control	Condición de aceptación
Integridad de recursos	Cero alertas missing-integrity y páginas funcionales
Backend	Sin alerta exec-use o con excepción técnicamente justificada
Cobertura	Sin autolectura del JSON y con Dockerfile e index.php revisados
Trazabilidad	JSON, captura, commit y decisión de triage conservados

Conclusión

El hallazgo accionable del escaneo es la falta de verificación de integridad del script remoto en ocho páginas públicas. Las alertas de backend no muestran una vía de explotación en el código revisado, aunque una mejora en verificar.php facilitará el cierre automático. El informe debe actualizarse después de aplicar las correcciones y repetir el análisis.

Revalidación posterior a correcciones

Semgrep OSS se ejecutó nuevamente el 21/10/2026 sobre el árbol de trabajo derivado del commit **5f15b3b**. La ejecución terminó correctamente y produjo un JSON independiente para comparar los resultados antes y después de las correcciones.

11	6	0	546
coincidencias iniciales	coincidencias posteriores	missing-integrity restantes	reglas ejecutadas

Comparación y decisión

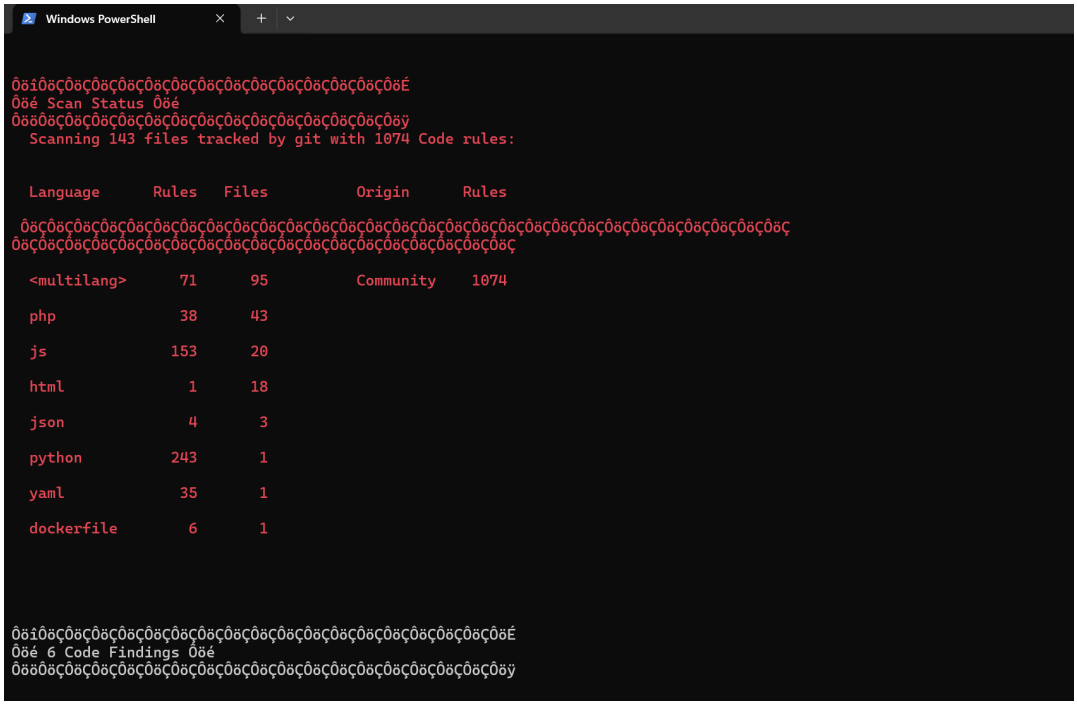
Control	Antes	Después	Decisión
Integridad de recursos	8 alertas por Lucide remoto sin integrity.	0 alertas; los iconos se sirven desde un archivo local.	Cerrado.
Ejecución de procesos	passthru con comando construido como texto.	proc_open recibe una lista de argumentos y no invoca un shell.	Mitigado; Semgrep conserva 1 alerta por el uso del sink.
Ruta de caché	1 alerta tainted-filename.	Permanece la coincidencia; la clave continúa reducida a SHA-256 y raíz fija.	Falso positivo documentado.
Borrado de archivos	1 alerta unlink-use.	Permanece la coincidencia; realpath y UPLOAD_ROOT restringen el destino.	Mitigado; mantener pruebas de rutas.
Formularios de acceso	No se informaba CSRF porque no declaraban POST.	3 alertas de una regla para plantillas Django.	No aplicable: HTML estático, API PHP y autenticación Bearer sin cookies.

Conclusión SAST

La debilidad confirmada de cadena de suministro quedó corregida. Las seis coincidencias posteriores no representan seis vulnerabilidades explotables: tres son la regla Django aplicada fuera de contexto, dos conservan controles compensatorios ya revisados y una corresponde a una utilidad CLI que ahora ejecuta PHP sin construir una línea de shell.

Evidencia y límites de cobertura

Captura 2. Resumen de la segunda ejecución de Semgrep proporcionada por el equipo.



Trazabilidad

Artefacto	SHA-256
informes-sast-post/semgrep-post.json	1A11FF607656121E675DE1EE7D31A0BCAA7BD6CD18B2A402ED27A87478FDDE59
informes-sast-post/semgrep-post-terminal.txt	F48B522056EB3C56A94DE8D0000C3432D4EDBC84792F7F5BAF20073CCFE66E63

Limitaciones

La ejecución registró 1 advertencia de análisis parcial sobre el Dockerfile. Además, Semgrep indicó que limitó el alcance a archivos seguidos por Git: el nuevo archivo local de iconos todavía no estaba versionado y el ajuste final del nombre de la configuración Apache ocurrió después de esta ejecución. El cierre formal debe identificar el commit final y, si se exige cobertura exacta de todo el corte, repetir Semgrep una vez versionados esos cambios.